

ОТЧЕТ ОБ АУДИТЕ

Приложение: Форма заявки (задания 3-6)

Студент: Буйвал Анна Андреевна

1. Защита от XSS (Cross-Site Scripting)

Описание уязвимости: XSS позволяет злоумышленнику внедрить вредоносный JavaScript-код в поля формы. Если вывести данные пользователя без обработки, скрипт выполнится в браузере администратора или другого пользователя.

Пример уязвимого кода:

данные выводятся как есть

```
echo "<td>" . $app['fullname'] . "</td>";
```

Метод защиты: Все выводимые данные пропускаются через функцию htmlspecialchars(), которая заменяет специальные символы HTML (<, >, ", ', &) на безопасные HTML-сущности.

Исправленный код:

все данные экранируются

```
echo "<td>" . htmlspecialchars($app['fullname']) . "</td>";
```

Где применяется в проекте:

- task5/form.php — при выводе ошибок: `<?= htmlspecialchars($error) ?>`
- task5/form.php — при подстановке значений в поля: `value="<?= htmlspecialchars($values['fullname']) ?>"`
- task5/admin.php — при выводе таблицы заявок: `<?= htmlspecialchars($app['fullname']) ?>`

2. Защита от SQL Injection

Описание уязвимости: SQL Injection позволяет злоумышленнику внедрить SQL-код через поля ввода. Например, если вставить `' ; DROP TABLE application; --` в поле ФИО, это может удалить таблицу.

Пример уязвимого кода:

данные напрямую вставляются в SQL-запрос

```
$sql = "INSERT INTO application (fullname) VALUES ('$fullname')";
```

```
$pdo->query($sql);
```

Метод защиты: Используются подготовленные запросы (prepared statements) с плейсхолдерами ?. Данные передаются отдельно от SQL-кода через метод `execute()`, поэтому они никогда не интерпретируются как часть SQL-команды.

Исправленный код:

подготовленный запрос

```
$stmt = $pdo->prepare("INSERT INTO application (fullname) VALUES (?)");  
$stmt->execute([$fullname]);
```

Где применяется в проекте:

- Все SQL-запросы используют `prepare()` + `execute()`
- `task5/form.php` — вставка и обновление заявок
- `task5/admin.php` — все операции с базой данных

3. Защита от CSRF (Cross-Site Request Forgery)

Описание уязвимости: CSRF позволяет злоумышленнику заставить пользователя выполнить нежелательное действие (например, удалить заявку) путём перехода по специально созданной ссылке.

Пример уязвимого сценария:

Злоумышленник отправляет администратору ссылку: `http://site.ru/admin.php?delete=1`. Если администратор перейдёт по ней, заявка удалится без подтверждения.

Метод защиты:

1. Для критических действий (удаление) добавлено JavaScript-подтверждение: `onclick="return confirm('Удалить?')"`
2. Используется HTTP-авторизация для админки — без неё невозможно выполнить действия
3. Сессионная авторизация для пользователей — редактировать заявку может только владелец

Исправленный код (admin.php):

подтверждение удаления через JavaScript

```
<a href="?delete=<?= $app['id'] ?>"
    onclick="return confirm('Удалить заявку #<?= $app['id'] ?>?')">
    Удалить
</a>
```

Исправленный код (form.php):

БЕЗОПАСНО: проверка сессии перед редактированием

```
if (isset($_SESSION['user_id'])) {
    $editId = $_SESSION['user_id'];
    // редактирование только своей заявки
}
```

4. Защита от Information Disclosure

Описание уязвимости: Information Disclosure — утечка информации о системе. Например, вывод внутренних ошибок PHP или SQL пользователю может раскрыть структуру базы данных.

Пример уязвимого кода:

УЯЗВИМО: вывод внутренней ошибки пользователю

```
$pdo = new PDO(...);
```

при ошибке PHP покажет логин, пароль, структуру БД

Метод защиты:

1. Обработка ошибок через try-catch с выводом пользователю только общего сообщения
2. Режим продакшена в PHP: `display_errors = Off` на сервере
3. Детали ошибок логируются на сервере, но не показываются пользователю

Исправленный код:

БЕЗОПАСНО: пользователь видит только общее сообщение

```
try {
    $pdo = new PDO(...);
```

```
} catch (PDOException $e) {  
    die("Ошибка подключения к БД. Попробуйте позже.");  
    // Детали: error_log($e->getMessage());  
}
```

Где применяется в проекте:

- task5/form.php — подключение к БД обёрнуто в try-catch
- task5/admin.php — все операции с БД через try-catch
- Пароли хранятся в хешированном виде (bcrypt), не в открытом

5. Защита от Include-уязвимости

Описание уязвимости: Include-уязвимость позволяет злоумышленнику подключить произвольный файл через параметр URL (например, `?page=../../etc/passwd`).

Метод защиты:

- В проекте **не используется** динамическое подключение файлов (`include $_GET['page']`)
- Все файлы подключаются статически с фиксированными путями
- Нет пользовательского ввода, влияющего на выбор подключаемого файла

6. Защита от Upload-уязвимости

Описание уязвимости: Upload-уязвимость позволяет загрузить вредоносный PHP-файл на сервер и выполнить его.

Метод защиты:

1. В проекте **отсутствует** функциональность загрузки файлов
2. Все данные передаются через текстовые поля формы
3. Даже если бы загрузка была — необходимо проверять MIME-тип, расширение и содержимое файла

Выводы

В результате аудита были выявлены и устранены следующие уязвимости:

Уязвимость	Статус	Метод защиты
XSS	Защищено	<code>htmlspecialchars()</code> на всём выводе
SQL Injection	Защищено	Prepared statements (PDO)
CSRF	Защищено	JS-подтверждение удаления, сессионная авторизация
Information Disclosure	Защищено	Try-catch, хеширование паролей
Include	Отсутствует	Статические пути, нет динамического include
Upload	Отсутствует	Функциональность не реализована